

Quest Coder PRD

Sep 30, 2026 · @Eric Weng

Overview

Quest Coder is a web app that turns each LeetCode or NeetCode problem into a short RPG story: two to eight mini quests that each solve one piece of the problem, ending in a boss battle against the full problem.

Practicing on LeetCode today means facing the whole problem at once. A failing test shows an input, an expected output, and a wrong answer, so you still have to work out why in your head. Quest Coder changes that in three ways:

- **Modular problems.** Each problem is split into quests that build on each other, so the boss fight is assembled from pieces you already understand.
- **Visible execution.** Every time you run code, you can watch it run against each test as an animation: what each variable holds, where it moves, and the exact line where it went wrong. You can tell at a glance whether it failed a test or didn't compile at all.
- **Spaced reinforcement.** Surprise battles and stale-question reviews bring back topics and problems before you forget them.

The visual standard is the "Two queues, one party" page built for Merge Two Sorted Lists, which showed the result building in real time while the `list1`, `list2`, and `node` markers moved with the data. Every boss fight has to meet that bar.

The Timequake Dojo (Search in Rotated Sorted Array) proved the core loop works: quests, tests, read budgets, and line-by-line replays, all in one page. Quest Coder turns that into a real app with accounts, saved progress, a question library, and a server that runs code.

Goals and non-goals

The goal is understanding that sticks: a player who beats a boss can explain the solution and solve it again weeks later.

Goals

1. Every question in the library is broken into 2 to 8 quests with a story, ending in a boss battle against the full problem.

2. Every code run can be watched as an animation against each test, showing variables, data movement, and the failing line.
3. A player can always tell the difference between a failed test, a crash, and code that didn't compile.
4. Progress, attempts, and stats are saved to the player's account.
5. Surprise battles and stale reviews bring back old topics and problems on a schedule.
6. Adding a question is a content task (writing a quest pack), not an engineering task.

Non-goals for this PRD

- Languages other than Python.
- Copying LeetCode or NeetCode problem text. Problems are retold in the app's own words and story (see Risks).
- Any real game's characters, art, or UI. The world is original, in a MapleStory-like spirit.

Deferred features (planned, but not specified here)

- The reward system: gold, EXP, items, or something else. Quests will grant a small reward and bosses a bigger one, but the currency and what it buys are still open.
- A stat bar showing the player's level and strengths by topic.
- A friend list and anything social.

Users and rollout

The first user is the builder, a new-grad engineer practicing for interviews; the app opens to the public once it holds up for that one player.

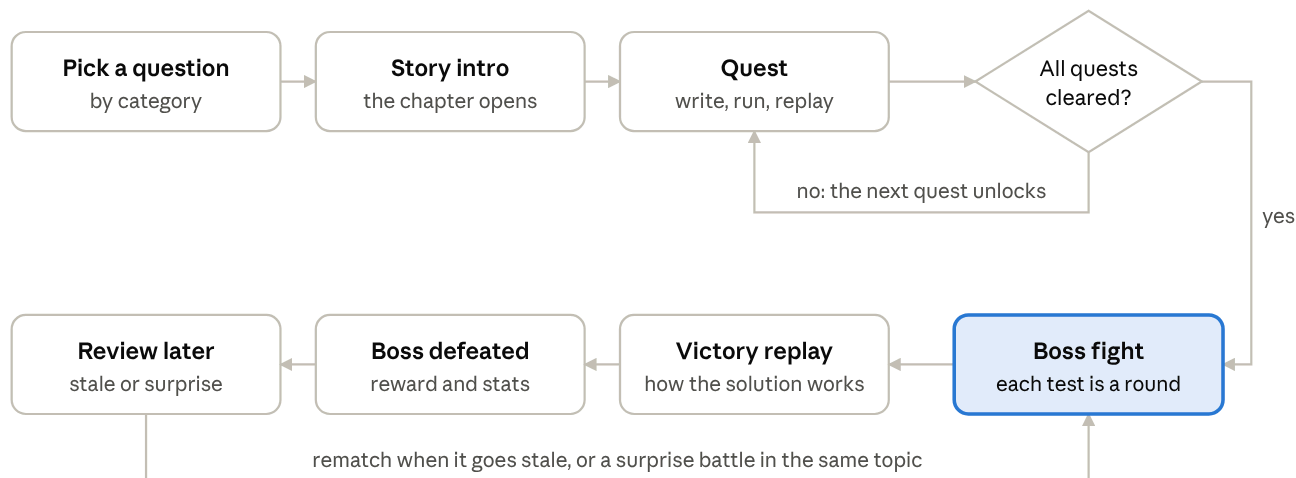
Stage	Who	What has to be true
Personal	The builder only	Login, saved progress, and the full quest-to-boss loop work for every question added so far
Public	Other interview-prep learners	Problem text is fully original, the code sandbox is hardened for strangers' code, and sign-up and accounts work at scale

The target player learns best from visuals and interaction, gets lost in a wall of problem text, and stays motivated when practice feels like a game. The builder adds questions one at a time, each submitted in chat and turned into a quest pack.

Core loop

Every question follows the same loop: story, quests one at a time, a boss fight, a victory replay, and later a review that brings the boss back.

Quests build up to the boss, and reviews bring the boss back



core loop · quests, boss, replay, review

Inside each quest and boss round, the player writes code, runs it, and watches the animation of any failing test before fixing and running again.

Quest packs

Each question ships as one quest pack: a data file holding its story, quests, tests, and visual spec, so the app can add a question without new code.

The builder submits a question in chat, Claude writes the pack, the pack is validated (every reference solution passes, every scene renders), and then it's loaded into the database.

Part	What it holds	Example (Search in Rotated Sorted Array)
Metadata	Title, category, difficulty, original problem number, prerequisites	Binary search, medium, LeetCode 33
Story	The world, the setting, and a short chapter for each quest and the boss	A timequake scrambled a hallway of monster rooms

Part	What it holds	Example (Search in Rotated Sorted Array)
Quests (2 to 8)	Brief, function signature, starter code, hints written as questions, reference solution, reward	Quest 3: is the left half smooth? 3 rocks
Tests	Fixed edge cases and generators for large random inputs, shared by every run	One room, two rooms, no rotation, missing target, 1,000 rooms
Budgets	Read limits that enforce the complexity target, per quest	About $6 \times \log_2(n)$ reads for the boss
Boss	The full problem with the real signature, its tests, and its replay script	<pre>class Solution: def search(...)</pre>
Scene spec	Which renderer draws the data (rooms, portals, islands, trees), and which variables to track	Doors up to 16 rooms, skyline above; track <code>l</code> , <code>mid</code> , <code>r</code>
Review set	Short variants used for surprise battles and stale reviews	Same idea with a different target or rotation

Quests unlock in order. The boss unlocks when every quest is cleared.

Boss fight and replay

The boss has two parts: you fight it with your own code, then you watch a victory replay that shows how the solution works. Both must make the algorithm obvious at a glance.

The fight. The boss is the full problem. Each test is one round. A passing test lands a hit on the boss; a failing test is the boss's counterattack, and that round's animation shows exactly why it failed. The boss is defeated when every test passes within budget.

The victory replay. After the win, the player watches the full solution run on a representative input, step by step, with the story's narration. This is the reinforcement moment: it should make the solution feel easy to explain.

Every fight round, quest run, and victory replay must meet this visual bar:

1. **Variables are characters.** Each tracked variable (`l`, `r`, `mid`, `node`, `slow`, `fast`, `a` stack top) is a visible marker or character that physically moves when its value changes.
2. **Data moves.** When a value moves (a node is relinked, an item is pushed, a pointer advances), the animation shows it travelling from where it was to where it goes.
3. **The code follows along.** A code panel highlights the line being executed at every step, with the current values of the tracked variables.
4. **Every outcome looks different.** Found, correct "not found," wrong answer, crash, off the end of the array, infinite loop, over the complexity budget, and "didn't compile" each have their own visual.
5. **Wrong answers show the right one.** On a wrong answer, the expected result is highlighted next to the player's result.
6. **It works on big inputs.** Large tests switch to a compact view (like the skyline of bars) that still shows pointers and progress.
7. **The player controls time.** Play, pause, step forward and back, skip to the end, and speeds from 0.5x to 4x.

The benchmark is the "Two queues, one party" page: the merged line built itself in real time, and every pointer tag rode on the player it pointed to. If a new question's visual is less clear than that, it doesn't ship.

Review system

Two kinds of review keep old material fresh: surprise battles reinforce a topic with a new question, and stale reviews bring back a question you've already beaten.

	Surprise battle	Stale review
What it is	An unannounced fight against a question you haven't played, in a topic you've studied	A rematch against a boss you've already defeated
When it appears	After you clear questions in a topic, at random intervals	When a beaten question hasn't been played for a while, on a spaced schedule
Source	Another question in the same category, or a review variant from a pack	That question's review set, so it isn't a word-for-word repeat

	Surprise battle	Stale review
Format	A single boss-style fight, quests skipped	The boss alone, with quests available if you get stuck
Result	Strengthens your rating in that topic	Resets the stale timer, with a longer gap after each win

Staleness uses spaced repetition: a question that's won easily comes back later each time, and one that's lost or needs hints comes back sooner. The exact intervals are an open question.

Functional requirements

P0 is needed for the personal stage, P1 for the public launch, and P2 is later.

ID	Area	Requirement	Priority
R1	Accounts	Sign up, log in, and log out; sessions persist across devices	P0
R2	Library	Browse questions by category (two pointers, stacks, graphs, greedy, dynamic programming, recursion, and more)	P0
R3	Library	Each question shows its story intro, quest count, difficulty, and your status (not started, in progress, defeated, stale)	P0
R4	Quests	2 to 8 quests per question, unlocked in order, each with a brief, signature, starter code, and question-style hints	P0
R5	Editor	Python editor with line numbers, auto-indent, Tab and Shift+Tab, a run shortcut, and font ligatures off	P0
R6	Running	Run code against the quest's tests on the server and show results in a few seconds	P0
R7	Running	Clearly separate four results: didn't compile, crashed, failed a test, passed	P0

ID	Area	Requirement	Priority
R8	Running	Enforce complexity with read budgets, and show used versus allowed	P0
R9	Running	Stop infinite loops and runaway memory with time and memory limits, and explain what happened	P0
R10	Animation	Every test of every run can be replayed to the visual bar in "Boss fight and replay"	P0
R11	Animation	The first failing test's animation opens automatically after a run	P0
R12	Boss	The boss fight plays each test as a round, then unlocks the victory replay	P0
R13	Progress	Save code per quest, every attempt, quests cleared, and bosses defeated	P0
R14	Progress	A hidden reference solution per quest, opened only on request and recorded as used	P0
R15	Review	Stale reviews scheduled with spaced repetition	P0
R16	Review	Surprise battles drawn from topics the player has studied	P1
R17	Stats	Track per-player and per-topic stats: questions defeated, attempts, hint and solution use, streaks	P1
R18	Content	Load a validated quest pack without changing app code	P0
R19	Content	Reject a pack whose reference solutions fail its own tests or whose scenes don't render	P0
R20	Public	Rate limits, abuse protection, and a hardened sandbox for untrusted users	P1
R21	Rewards	Small rewards per quest and a bigger one per boss (design to come)	P2
R22	Social	Stat bar and friend list	P2

Data model

Content tables hold the library and are written only by loading quest packs; player tables hold everything a player does.

Content

Table	Key fields	Belongs to
<code>categories</code>	id, name (two pointers, stacks, graphs, greedy, DP, recursion, ...), display order	—
<code>questions</code>	id, category_id, title, difficulty, source problem number, story intro, scene spec, pack version	a category
<code>quests</code>	id, question_id, order (1 to 8, last one is the boss), brief, signature, starter code, hints, reference solution, read budget, reward	a question
<code>test_cases</code>	id, quest_id, label, inputs, expected output, kind (fixed or generated), generator seed	a quest
<code>review_variants</code>	id, question_id, inputs and tests for surprise battles and stale reviews	a question

Player

Table	Key fields	Belongs to
<code>users</code>	id, email, password hash or OAuth id, display name, created at	—
<code>quest_progress</code>	user_id, quest_id, status (locked, open, cleared), saved code, hints opened, solution viewed	a user and a quest
<code>attempts</code>	id, user_id, quest_id, code, result (didn't compile, crashed, failed, passed), tests passed, reads used, run timeline, created at	a user and a quest
<code>question_progress</code>	user_id, question_id, status (in progress, defeated), defeated at, last played, next review at, review interval	a user and a question

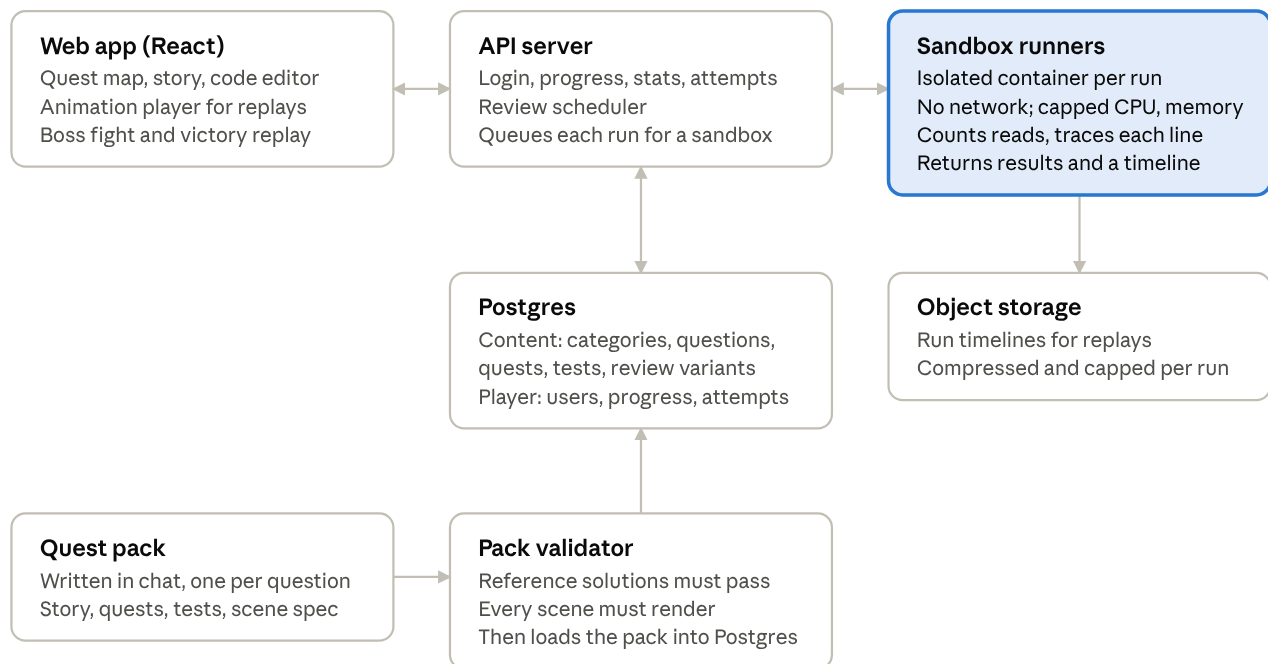
Table	Key fields	Belongs to
<code>topic_stats</code>	<code>user_id</code> , <code>category_id</code> , questions defeated, attempts, hint use, rating	a user and a category
<code>battles</code>	<code>id</code> , <code>user_id</code> , <code>question_id</code> , kind (surprise or stale), result, created at	a user and a question

Two storage notes: test cases generated from seeds can be rebuilt instead of stored in full, and run timelines can grow large, so they may go in object storage with only a pointer in `attempts`.

Architecture

The browser never runs player code: the API sends every run to an isolated sandbox, which returns pass or fail results plus a timeline the web app animates.

The API hands every run to an isolated sandbox and stores what comes back



architecture · app, API, sandbox, storage, content pipeline

Key design decisions:

- **Tracing in real Python.** The sandbox runs CPython, so line tracing can use `sys.settrace` and real local variables, which is simpler and more accurate than the in-browser approach the dojo needed.
- **Two passes per run.** Tests run untraced first for a fast pass or fail. Only the tests being replayed are traced, so tracing never causes a timeout.
- **Timelines are the contract.** The sandbox outputs a timeline of steps (line, variable values, reads, data moves), and the scene renderer only consumes timelines. New scenes need no sandbox changes.
- **Read counting.** Inputs are wrapped in counting objects, as in the dojo, so complexity budgets stay exact across machines.
- **Packs are validated before loading.** The validator runs every reference solution in the same sandbox, so a broken pack never reaches players.

Risks and mitigations

The two biggest risks are running strangers' code safely and making a great visual for every single question.

Risk	Impact	Mitigation
Untrusted code on the server	Someone escapes the sandbox or attacks other users	Run each job in an isolated container or microVM with no network, a read-only filesystem, and CPU, memory, and time limits; harden this before going public
Visual quality varies by question	Some bosses teach less than others, which breaks the core promise	One visual bar for every pack (see "Boss fight and replay"); review each pack's replay before it's published
Each question takes real work to author	The library grows slowly	Reusable scene renderers per data structure (arrays, linked lists, trees, graphs, stacks), so a new pack mostly reuses an existing scene

Risk	Impact	Mitigation
Problem text is copyrighted	Public launch could face takedowns	Retell every problem in original words inside the story; reference the problem only by name and number
Server runs feel slower than in-browser runs	Waiting breaks the flow of trying things	Keep the job queue short, reuse warm containers, and show progress while tests run
Line tracing makes code slower	Large tests hit the time limit only because of tracing	Run tests untraced for pass or fail, then trace only the tests being replayed
Run timelines are large	Storage costs grow with every attempt	Cap timelines, compress them, and keep full timelines only for recent attempts
Review schedule feels random or nagging	Players ignore surprise battles	Let players see what's coming up and snooze it; tune intervals with real use

Roadmap

The engine comes first, because the animations are the product: once a boss runs on the server with dojo-quality replays, the rest of the app is built around it. No dates are set yet.

Prove the engine on the server first, then build the app around it

Phase 1 · Engine on the server (next)

Sandbox runner with read counting and line tracing; the timeline format
Port the dojo's replay player and the rooms scene to the web app



Gate: the Timequake boss runs on the server with the same replays as the dojo

Phase 2 · Personal MVP

Login, category library, quest packs, saved code, attempts, and progress
Boss fights and victory replays; linked-list and rooms scenes



Gate: you clear 5 questions end to end in the app

Phase 3 · Reinforcement

Stale reviews on a spaced schedule, surprise battles, and topic stats
More scenes: stacks, trees, and graphs



Gate: a sandbox security review passes and every problem text is original

Phase 4 · Public launch

Hardened sandbox, rate limits, and sign-up for everyone
Then the deferred features: rewards, stat bar, and friend list

roadmap · 4 phases, 3 gates, no dates set

The questions already worked through in chat (Reverse Linked List, Merge Two Sorted Lists, Linked List Cycle, and Search in Rotated Sorted Array) are the natural first quest packs.

Open questions

- ☐ What's the app's real name? "Quest Coder" is a placeholder.
- ☐ What do quests and bosses reward (gold, EXP, items), and what can the player spend it on?
- ☐ Should a failed boss round cost something, such as hit points that reset the fight, or is it free to retry?
- ☐ How often should surprise battles appear, and what intervals should stale reviews use?
- ☐ Does viewing a reference solution lower the reward or block the boss until a later retry?
- ☐ Which questions go in the first library, and should it follow the NeetCode 150 order?

- ☐ Is the tech stack open, or should it match what you already use (React, TypeScript, Postgres)?
- ☐ Should quest packs also be exportable as standalone dojo pages, like the Timequake Dojo?